RapidFireAI Context Engineering

Chia-Han Chen
Department of Computer Science and Engineering
University of California San Diego
La Jolla, CA 92092
chc016@ucsd.edu

Emily Chen
Halıcıoğlu Data Science Institute
University of California San Diego
La Jolla, CA 92092
emc001@ucsd.edu

1 Data Creation Methodology

For our first trial, we gave a short prompt to ChatGPT 5.5 (thinking mode, extended effort) that included the structure and format of the original 45 golden Q&A pairs as a reference. We diversified the question set by persona and domain-driven purposes to cover a broader range of how a real user might query the documentation. We deliberately generated more candidate questions than required, so we could discard any that did not meet our golden QA standards (e.g., ambiguous phrasing, answers not fully grounded in the docs, or duplicate coverage). After manual review, we removed the guiding persona framing to produce more direct Q&A pairs. We also added multi-file synthesis questions that require combining information from 2–4 distinct `.rst` files, incorporating 7

Then, we ensured quality assurance by running through Claude Opus 4.6 high thinking mode to verify that reference answer were indeed written from the source, line numbers actually contained the evidence.

2 Final Pipeline Architecture

2.1 Text Loader

To load the documents, we wrote a text loader inheriting from the LangChain text loader as the original `TextLoader` does not allow us to have a metadata function. We treat the `rst` files as plain text files and use the `TextLoader` from LangChain to load the content. The only difference is that in `lazy_load` where we allow a metadata function to write metadata to the documents, which is used by `DirectoryLoader`.

We had also tried `UnstructuredRSTLoader` as the documents are stored as `rst` files and we expected that using a loader specifically designed for `rst` files could help with structure capturing. However, it could not properly read the TeX format and a warning would show up as the following when there is math in the document.

> [WARNING] Could not convert TeX math

Therefore, to avoid the warnings, we decided to treat the documents just as plain text and use `TextLoader` instead.

2.2 Chunking Strategy

For chunking, we also wrote a text splitter inheriting the `RecursiveCharacterTextSplitter` from LangChain to not only split the documents into chunks but also write the start and end line numbers into the metadata. In `split_documents` function, we split each document normally and create a index mapping for each line. Then, for each chunk, we assign the corresponding start and end line numbers to the chunk and store into the metadata.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

To split the documents, we use `from_tiktoken_encoder` to chunk the documents using token count instead of character count. We used the `o200k_base` model, which is OpenAI's tokenizer for GPT-4o.

For the configuration, we use chunk size of 384 with overlap of 64 and set the separators to the new line character "`\n`" and "`\n\n`".

2.3 Embedding Model

For the embeddings, we use `api-tgpt-embeddings` from OpenAI to embed the documents and the quieries as it is free and it outperformed the other embedding models we experimented with in Section 3.

2.4 Retrieval Method

For the retriever, we use RapidFire AI default retriever, which is `BaseRetriever` from LangChain to help us get the relevant documents.

For vector storage, we use `FAISS` as it does not require any additional api key like `Pinecone` and does not need a database connection like `Postgres PGVector`.

For the searching algorithm, we use maximal marginal relevance (MMR) with k being 15 to search for the most relevant documents.

2.5 Reranking

We use `CrossEncoderReranker` with `BAAI/bge-reranker-v2-m3` to rerank the retrieval. We set the `top_n` to 8 to reorder the retrieved chunks.

3 Experimentation Methodology

3.1 Chunk Size Experiments

For chunk size, we started with chunk size of 512 and overlap of 64 as the baseline to compare to. With chunk size of 512, each chunk contains decent amount of information and the overlap maintains the context continuity across splits.

Our experiments contained two parts, increasing the chunk size and decreasing the chunk size. We experimented with chunk size of 384, 512, and 1024. The purpose of increasing the chunk size is to allow each chunk to contain more information ideally, but the tradeoff is that the information could be more noisy and more context could be wasted. We also experimented with smaller to chunk size since ideally the information could be more dense, but the information could be more fragmented and thus each chunk might not be able to contain full and completed information.

3.2 Retriever Experiments

For the retriever, we experimented with two kinds of retrievers from LangChain, `TFIDFRetriever` and `BaseRetriever`, which is also the default retriever used by RapidFire AI to evaluate how different retrieval strategies impact the performance.

The goal was to see the tradeoffs between semantic retrieval, which captures contextual meaning through embeddings, and keyword-based retrieval, which relies on exact term matching.

3.3 Embedding Experiments

For the Hugging Face embeddings, we tried `sentence-transformers/all-MiniLM-L6-v2` and `BAAI/llm-embedder` where `sentence-transformers/all-MiniLM-L6-v2` is a smaller model that maps sentences and paragraphs to a 384 dimensional dense vector space, and `BAAI/llm-embedder` is a unified embedding model to support diverse retrieval augmentation needs for LLMs

For the Open AI embeddings, we tested `api-tgpt-embeddings`, which is an OpenAI's API embeddings that is free to us.

3.4 Searching Algorithm Experiments

The experiments for vector search contains two parts. One is comparing the type of search, and the other is the number of documents to retrieve which is k.

For the search type, we tried cosine similarity based search and Maximum Marginal Relevance (MMR) search to see the difference between these two metrics for searching. For k, we tried multiple k's and expected that as the number of retrieved documents increases, the recall should also increase.

3.5 Reranker Experiments

For the reranker, we experimented with different cross encoder reranker including `cross-encoder/ms-marco-MiniLM-L6-v2`, `cross-encoder/ms-marco-MiniLM-L12-v2`, and `BAAI/bge-reranker-v2-m3`. We also tried different configurations of `top_n` to see if retrieving more documents help.

3.6 LLM Model Experiments

We tried three LLM model APIs for generating answers, `api-mistral-small-3.2-2506`, `api-gemma-4-26b`, and `api-gpt-oss-120b`.

Ideally, GPT OSS 120B should provide the strongest reasoning over the retrieved context, but it is also the most expensive model and has the highest latency among these three models. Given the moderate size, Gemma 4 26B should offer a balance of generation capability and cost. As the smallest model, Mistral Small 3.2 should be a fast and low-cost baseline, which can be helpful in high-throughput scenarios.

3.7 Eight Meaningful Configurations

Full sweeps produced many near-tied configs, but these eight are selected to cover distinct trade-off regions rather than repeating near-duplicate settings.

#	Rnd	Generator	Chunk	Embed	Search	Reranker	Ret./Gen. Scores
1	R1	mistral-small	512	OpenAI	mmr k=10	L6-v2 (n=5)	0.450 / 0.902
2	R2	gpt-oss-120b	512	OpenAI	sim k=10	L12-v2 (n=5)	0.408 / 0.939
3	R3	gpt-oss-120b	384	OpenAI	mmr k=15	bge-v2-m3 (n=8)	**0.542** / 0.942
4	R3	gpt-oss-120b	512	OpenAI	mmr k=18	bge-v2-m3 (n=8)	0.515 / **0.967**
5	R4	llama-4-scout	512	mpnet	sim k=15	L6-v2 (n=5)	0.483 / 0.909
6	R5	gpt-oss-120b	512	OpenAI	mmr k=15	bge-v2-m3 (n=10) + TF-IDF	0.172 / 0.881
7	R6	gpt-oss-120b	512	OpenAI	mmr k=18	bge-v2-m3 (n=10)	0.491 / 0.967
8	R7	gpt-oss-120b	512	OpenAI	mmr k=20	bge-v2-m3 (n=12)	0.505 / 0.957

4 Results and Analysis

These eight are selected to cover distinct trade-off regions rather than repeating near- duplicate settings.



Configs using api-gpt-oss-120b with OpenAI api-tgpt-embeddings, MMR (k=15–20), and bge-reranker-v2-m3 (top_n=8–12) performed best, while TF-IDF/sparse retrieval and many gemma-4-26b configurations performed worst. This is because dense semantic retrieval plus stronger reranking and larger candidate pools consistently found better evidence, while sparse retrieval and weaker format-following hurt source extraction and overall quality. Retrieval was most sensitive to embedding choice, retriever type, and search breadth (k, top_n), whereas generation was most sensitive to generator model capacity and then reranker/context quality. It was surprising that instruction-tuned BAAI/llm-embedder still trailed OpenAI embeddings and that gemma-4-26b sometimes had decent judge scores but poor retrieval because of brittle source-format compliance. With more time and budget, we would run multi-seed experiments with confidence intervals, test adaptive/hybrid retrieval policies (dynamic k/top_n), and further harden prompt/output formatting plus domain-specific embedding fine-tuning.

5 AI Tools Statement

AI assistants were used throughout the project for learning, debugging, and development:

ChatGPT, Claude, and Cursor, were used for understanding API options (tiktoken encoding names, LangChain retriever types), debugging RapidFire AI import issues (RFOpenAIAPIModelConfig returning None due to vLLM dependency), and general Python/LangChain guidance.

Used for code exploration, debugging runtime errors, understanding the evaluation pipeline, and generating visualizations.

All AI-generated suggestions were reviewed, tested, and adapted to our specific codebase. The golden Q&A pairs were written by reading the source documentation directly and AI was used to generate reference answers.